

0.93.2. Un jeu de bombes — A pose une bombe

Le 0.93.1, et le bouton A pose une bombe B sous le O. Une seule à la fois. Elle ne se voit que quand le O s'en va ; ensuite, elle lui barre le chemin.

1. Ce qu'il faut comprendre

C'est le 0.93.1, avec une chose en plus : la bombe. Le bouton **A** pose une bombe, une **B**, là où est le O. Pour l'instant, elle n'explose pas : ce sera le 0.93.3.

Trois variables : `bombe` vaut 1 quand une bombe est posée, 0 sinon ; `bx` et `by` retiennent sa place.

Poser : `if (bouton(A) && bombe == 0) ( &&, le 0.75). bombe == 0 : une seule bombe à la fois. Tant qu'elle est là, A ne fait plus rien. Alors bombe = 1;, et la bombe prend la place du O : bx = ox; by = oy;.`

On ne la voit pas tout de suite : le O est encore dessus, et c'est lui qu'on voit. **Ce qui est nouveau ici : la B apparaît quand le O s'en va.** Au moment du pas, on efface la case du O, puis : `if (bombe == 1 && ox == bx && oy == by)` : si le O était **sur** la bombe, on pose la B à cette place. Sans ça, `effacer()` effacerait aussi la bombe.

Ensuite, la bombe barre le chemin. Au 0.93.1, le O allait partout sauf sur un X. Il faudrait maintenant dire « ni un X, ni une B »... et bientôt « ni un W, ni une flamme ». Plus simple : **le O ne va que sur une case VIDE.** Une case vide, c'est un espace, la **tuile 0** : `lire(nx, ny) == 0`. X, B, et tout ce qui viendra plus tard, l'arrêtent d'un coup.

Déroulons : O en (3, 1), A : `bombe = 1, bx = 3, by = 1`. DROITE : `lire(4, 1)` vaut 0, vide. On efface (3, 1) ; le O était sur la bombe (3 == 3 et 1 == 1) : la B est posée en (3, 1). Le O va en (4, 1). GAUCHE : `lire(3, 1)` rend la B, pas 0 : le O ne bouge pas.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **774 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          72 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
main()             344 o ← démarrage — ta fonction
```

VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
CopierTuiles	49 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	48 o	← CopierTuiles – 3 lettres sur 43

Lettres gravées : BOX

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 5	65 o sur place, dans main() + AdresseCase (10 o)
bouton()	× 5	25 o sur place, dans main() + LireManette (37 o)
effacer()	× 1	15 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
lire()	× 1	11 o sur place, dans main() + AdresseCase (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	× 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN JEU DE BOMBES ----
// Les lettres : X = un mur, O = le joueur, B = une bombe, W = un ennemi.
// ALPHABET[23] : le X    ALPHABET[14] : le O
// ALPHABET[1]  : le B    ALPHABET[22] : le W

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
```

2. UNE VARIABLE — OX

```
uint8_t ox = 1;          // le O, le joueur : sa colonne... (le 0.93.1)
```

3. UNE VARIABLE — OY

```
uint8_t oy = 1;          // ...et sa ligne. Il part du coin (1, 1).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — BOMBE

```
uint8_t bombe = 0;       // NOUVEAU : 1 : une bombe est posée ; 0 : pas de bombe
```

7. UNE VARIABLE — BX

```
uint8_t bx = 0;          // la place de la bombe : sa colonne...
```

8. UNE VARIABLE — BY

```
uint8_t by = 0;           // ...et sa ligne
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- Le terrain : le cadre et les piliers X (le 0.93) ----
    for (uint8_t l = 0; l < 17; l++) {
        for (uint8_t c = 0; c < 19; c++) {
            if (l == 0 || l == 16 || c == 0 || c == 18) {    // le cadre
                poser(c, l, ALPHABET[23]);
            }
            if (c % 2 == 0 && l % 2 == 0) {                  // les piliers : colonne ET ligne paires
                poser(c, l, ALPHABET[23]);
            }
        }
    }

    poser(ox, oy, ALPHABET[14]);           // le 0 à sa place (le 0.93)

    while (true) {
        image();

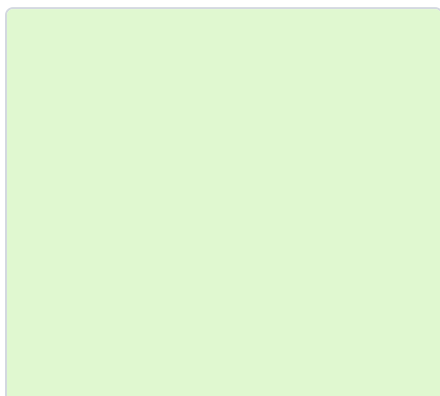
        // ---- le 0 bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
        if (chaque(150)) {
            // 1. la case d'ARRIVÉE : on part de la case actuelle (le 0.82).
            //    else if : une seule flèche à la fois, pas de pas en diagonale.
            nx = ox;
            ny = oy;
            if (bouton(DROITE)) {
                nx = ox + 1;
            } else if (bouton(GAUCHE)) {
                nx = ox - 1;
            } else if (bouton(BAS)) {
                ny = oy + 1;
            } else if (bouton(HAUT)) {
                ny = oy - 1;
            }

            // 2. une flèche est tenue (l'arrivée n'est pas la case où l'on est)...
            if (nx != ox || ny != oy) {
                // 3. NOUVEAU : ...et l'arrivée est VIDE : ni X, ni B.
                //    lire() rend 0 pour une case vide : c'est l'espace, la tuile 0.
                if (lire(nx, ny) == 0) {
                    effacer(ox, oy, 1);           // le 0 quitte sa case...
                    if (bombe == 1 && ox == bx && oy == by) {
                        poser(bx, by, ALPHABET[1]); // NOUVEAU : ...mais s'il était sur la bombe, la B apparaît
                    }
                    ox = nx;                       // ...et va sur la case d'arrivée
                    oy = ny;
                    poser(ox, oy, ALPHABET[14]);
                }
            }
        }

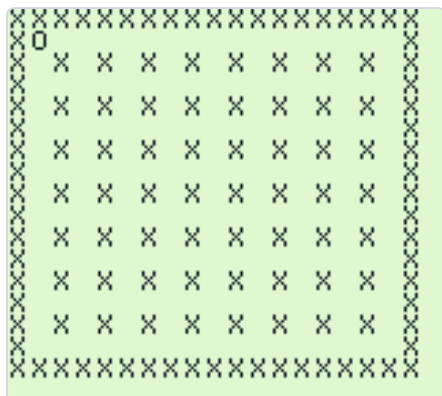
        // ---- NOUVEAU : A : poser une bombe, là où est le 0 ----
        // bombe == 0 : une seule bombe à la fois.
        if (bouton(A) && bombe == 0) {
            bombe = 1;
            bx = ox;                               // la bombe est sous le 0
            by = oy;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



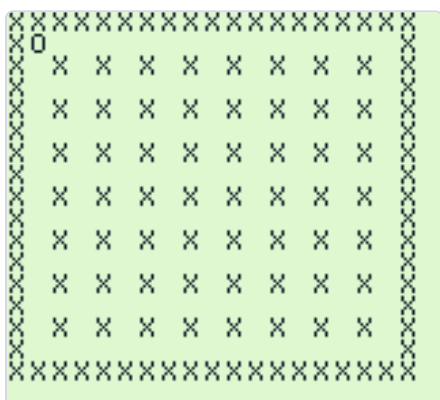
la toute première image — la console vient de démarrer — image 1



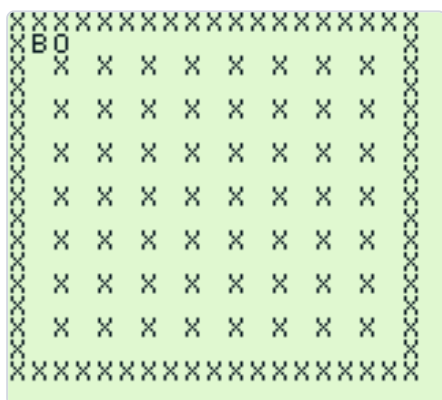
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

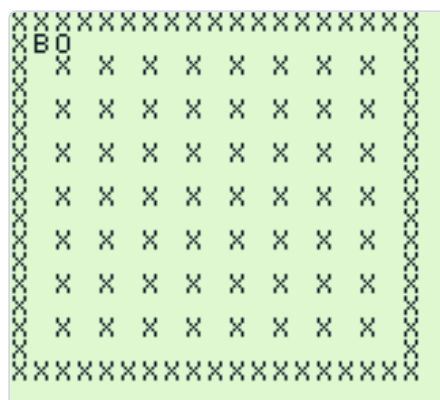
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



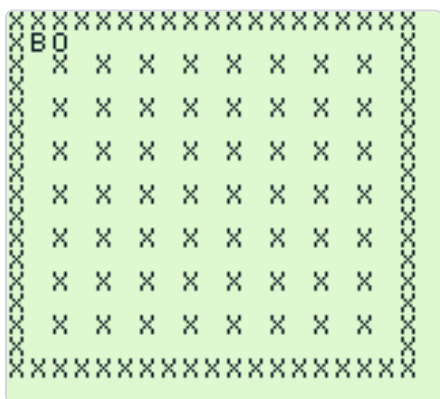
A — l'écran n'a pas bougé



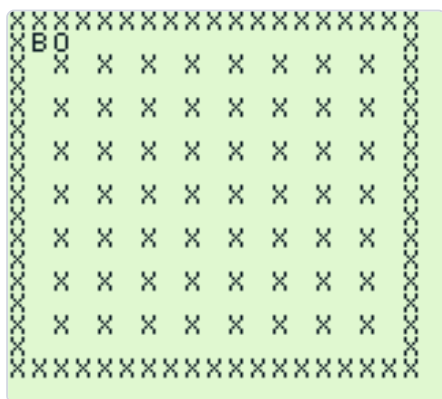
DROITE — l'écran a changé



GAUCHE — l'écran n'a pas bougé



HAUT — l'écran n'a pas bougé



BAS — l'écran n'a pas bougé

5. Ce qu'on doit voir

A pose une B sous le O ; elle apparaît quand il s'en va, et il ne peut plus repasser dessus.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ A : une bombe en (3, 1)

✓ le O parti, la B apparaît : B O

✓ la B barre le chemin : le O reste en (4, 1)

✓ une seule bombe à la fois

Ensuite : **0.93.3. Un jeu de bombes — la bombe explose** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++